



TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
PULCHOWK CAMPUS

A
PROGRESS REPORT
ON
HANDWRITTEN MATHEMATICAL PROBLEM SOLVER
(IPCV)

SUBMITTED BY:

SANDIP KATEL (078BCT077)
SAPHAL RIMAL (078BCT078)
SHARAD POKHARAL (078BCT083)
SIJAN JOSHI (078BCT087)

SUBMITTED TO:

DEPARTMENT OF ELECTRONICS & COMPUTER ENGINEERING

Baisakh, 2082

Page of Approval

TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
PULCHOWK CAMPUS
DEPARTMENT OF ELECTRONICS AND COMPUTER ENGINEERING

The undersigned certifies that they have read and recommended to the Institute of Engineering for acceptance of a project report entitled ”**Handwritten Mathematics Problem Solver**” submitted by **Saphal Rimal, Sandip Katel, Sharad Pokharel** and **Sijan Joshi** in partial fulfillment of the requirements for the Bachelor’s degree in Electronics & Computer Engineering.

.....

External examiner

Assistant Professor

Department of Electronics and Computer Engineering,
Pulchowk Campus, IOE, TU.

Date of approval:

Copyright

The author has agreed that the Library, Department of Electronics and Computer Engineering, Pulchowk Campus, and Institute of Engineering may make this report freely available for inspection. Moreover, the author has agreed that permission for extensive copying of this project report for scholarly purposes may be granted by the supervisors who supervised the work recorded herein or, in their absence, by the Head of the Department wherein the project report was done. It is understood that recognition will be given to the author of this report and the Department of Electronics and Computer Engineering, Pulchowk Campus, Institute of Engineering for any use of the material of this project report. Copying publication or the other use of this report for financial gain without the approval of to the Department of Electronics and Computer Engineering, Pulchowk Campus, Institute of Engineering, and the author's written permission is prohibited.

Request for permission to copy or to make any other use of the material in this report in whole or in part should be addressed to:

Head
Department of Electronics and Computer Engineering
Pulchowk Campus, Institute of Engineering, TU
Lalitpur, Nepal.

Acknowledgments

We extend our heartfelt gratitude to the Department of Electronics and Computer Engineering at Pulchowk Campus, Institute of Engineering. Our sincere thanks to the departmental leadership and the Project Management Team for their support and guidance. This collaborative project has been an invaluable opportunity to apply our academic knowledge and gain hands-on experience in teamwork. We are deeply appreciative of the departmental support that enabled us to undertake this meaningful Minor Project for our Third Year.

Special recognition goes to our seniors who generously shared their insights and expertise. We also thank our friends who contributed to this project in various ways, both directly and indirectly. Our deepest appreciation is reserved for our family members, whose constant inspiration and support have been crucial to our success. We welcome any suggestions or constructive criticism to further improve our work.

Authors:

Sandip Katel

Saphal Rimal

Sharad Pokharel

Sijan Joshi

Abstract

The project aims to develop a system for recognizing and solving handwritten mathematical expressions using TrOCR and SymPy. The system efficiently converts handwritten mathematical equations into digital text using TrOCR, a Transformer-based Optical Character Recognition (OCR) model. The recognized expressions are then processed with SymPy, a symbolic computation library, to generate real-time solutions. At its current stage, the model is capable of accurately interpreting diverse handwriting styles and mathematical notations across various domains such as algebra, calculus, differentiation, and simplification along with graph for visualizations of problem being solved. Future improvements will focus on optimizing performance, enhancing accuracy with better datasets, and integrating additional features to improve user experience and computational efficiency.

Contents

Page of Approval	ii
Copyright	iii
Acknowledgement	iv
Abstract	v
Contents	viii
List of Figures	ix
List of Abbreviations	x
1 Introduction	1
1.1 Background	1
1.2 Problem Statements	2
1.3 Objectives	3
1.3.1 Primary Objective	3
1.3.2 Specific Objectives	3
1.4 Scope	3
2 Literature Review	5
2.1 Related Work	5
2.2 Related Theory	6
2.2.1 Optical Character Recognition for Mathematical Notation	6
2.2.2 Mathematical Expression Parsing and Evaluation	7
3 Methodology	9
3.1 Feasibility Study	9
3.2 Requirement Analysis	9
3.3 Data Collection and Preprocessing	10
3.4 Model Development	10
3.4.1 Handwriting Recognition	10

3.4.2	Equation Parsing	10
3.4.3	Problem Solving	11
3.5	Development of the Web Application	11
3.5.1	Frontend Development	11
3.5.2	Backend Development	12
3.6	Integration	12
3.7	Testing and Evaluation	12
4	Experimental Setup	13
4.1	Data Collection and Preprocessing	13
4.1.1	InkML to Image Conversion	13
4.1.2	Data Cleaning and Augmentation	13
4.1.3	Dataset Splitting	13
4.2	Model Training Pipeline	14
4.2.1	Training Environment	14
4.2.2	Training Procedure	14
4.2.3	Evaluation Metrics	15
5	System Design	16
5.1	Context Diagram	16
5.2	Use Case Diagram	17
5.2.1	Figure	17
5.2.2	Description	17
5.3	System Architecture	18
5.3.1	Model Training Module	18
5.3.2	Character Recognition Module	20
5.3.3	Solution Generation Module	20
6	Results & Discussion	21
6.1	Transformation of Handwritten Expressions to solution	21
6.2	User Interface	22
6.3	Visualization	22
6.4	Statistical Analysis and Calculations	23
6.5	Test Results	24
6.6	Inference Time	24
6.7	Discussion	24

7	Conclusion	26
8	Limitations	27
8.1	Limitations of the Detection Section	27
8.2	Limitations of the Solving Section	28
9	Future Enhancements	29
9.1	Future Enhancements to the Detection Section	29
9.2	Future Enhancements to the Solving Section	30
	References	31

List of Figures

2.1	TrOCR Architecture Taken From [1]	7
3.1	Methodology - Handwritten Mathematical Problem Solver	9
5.1	Context Diagram for Handwritten Mathematical Problem Solver	16
5.2	Use Case Diagram for Handwritten Mathematical Problem Solver	17
5.3	Detailed System Architecture of the Handwritten Mathematical Problem Solver	19
6.1	Integration Problem: Handwritten, Recognized, and Solution	21
6.2	Integration Problem: Handwritten, Recognized, and Solution	21
6.3	Derivative Problem: Handwritten and Recognized	21
6.4	UI of the Web Application	22
6.5	Graph visualization of problem provided in user interface	22
6.6	Training process visualization: (a) shows the loss decreasing over training steps, (b) depicts the learning rate schedule, and (c) illustrates gradient norm variations.	23

List of Abbreviations

TrOCR	Transformer-based Optical Character Recognition
OCR	Optical Character Recognition
NLP	Natural Language Processing
CNN	Convolutional Neural Network
SymPy	Python library for Symbolic Mathematics
CROHME	Competition on Recognition of Online Handwritten Mathematical Expressions)
InkML	Ink Markup Language
JPEG	Joint Photographic Experts Group
RQ	Recognition Query
ViT	Vision Transformer
CER	Character Error Rate
WandB	Weights and Biases

1. Introduction

1.1 Background

The landscape of mathematical problem-solving stands at a critical technological crossroads, where the enduring tradition of handwritten notation encounters the increasingly sophisticated domain of computational interpretation. For generations, mathematicians, students, and researchers have relied on handwritten expressions as the primary medium for capturing complex mathematical ideas, developing unique notational styles reflecting individual cognitive processes. This reliance on handwriting stems from its flexibility, expressiveness, and direct connection to human thought processes.

However, existing digital technologies have struggled to effectively bridge the gap between human-generated mathematical notation and machine-based understanding. Optical Character Recognition (OCR) systems, despite significant advancements in text and image recognition, consistently falter when confronting the intricate nuances of mathematical symbols and their spatial arrangements. These challenges arise from the inherent complexity of mathematical notation, a language that transcends simple character recognition and demands sophisticated semantic understanding and structural analysis. Mathematical notation is not merely a collection of symbols but a complex linguistic system with contextual variations, cultural influences, and discipline-specific representations. Traditional digital tools often fail to capture the subtle contextual relationships that imbue mathematical expressions with deeper meaning.

Mathematics remains fundamental across numerous disciplines, including engineering, scientific research, and technological innovation. Yet, students and professionals continue to encounter significant technological barriers in digitizing their mathematical thinking. Handwriting persists as the preferred method of mathematical expression due to its intuitive cognitive processing and unparalleled flexibility in notation, enabling users to rapidly and naturally express complex ideas. The limitations of current digital tools impede productivity, hinder collaboration, and create obstacles in educational settings.

The proposed Handwritten Mathematics Problem Solver emerges as a transformative solution to these longstanding challenges. By integrating advanced machine learning techniques and computer vision algorithms, the project aims to create a seamless bridge between traditional handwritten practices and cutting-edge computational technologies. This innovative approach addresses critical gaps in current technological capabilities. It promises

to democratize mathematical problem-solving, reduce technological barriers in educational settings, and support diverse learning and working styles. The solution goes beyond mere character recognition, seeking to understand the semantic and contextual nuances of mathematical expressions, thereby enabling accurate interpretation, manipulation, and solution of handwritten mathematical problems. The broader significance of this project extends far beyond technical innovation. It represents a fundamental reimagining of how we interact with mathematical knowledge, transforming handwritten notation from a potential barrier to a gateway of computational understanding. By creating a system that can accurately interpret and solve handwritten mathematical problems, we open new possibilities for learning, research, and technological collaboration. At its core, the Handwritten Mathematics Problem Solver is more than a technological tool. It is a bridge between human creativity and computational power, promising to make mathematical problem-solving more accessible, efficient, and intuitive for students, educators, and professionals across various disciplines.

1.2 Problem Statements

In recent years, digital solutions for automating mathematical problem-solving have gained significant attention. Despite this progress, handwritten mathematical problems still pose challenges due to the variability in handwriting styles, non-uniform formatting, and complex symbols. Students, teachers, and researchers often face difficulties in digitizing and solving handwritten mathematical expressions efficiently.

Mathematics, particularly calculus, plays a vital role in various fields such as engineering, science, and technology. While digital tools have simplified the process of solving mathematical problems, typing complex calculus expressions remains a significant challenge for students, educators, and professionals. Many learners prefer writing these expressions by hand, especially during lectures, brainstorming sessions, and problem-solving tasks. However, existing digital tools often struggle to recognize and process handwritten problems effectively.

Thus, there is a pressing need for a Handwritten Mathematics Problem Solver that can accurately recognize, interpret, and solve handwritten mathematical problems. Such a system would alleviate the challenges of typing complex expressions and provide users with a seamless transition from traditional handwritten methods to digital computation. By addressing these issues, the tool would significantly improve accessibility, efficiency, and the overall learning experience.

1.3 Objectives

1.3.1 Primary Objective

Develop an application that can detect, digitize, and solve handwritten mathematical equations in real time with high accuracy and efficiency.

1.3.2 Specific Objectives

1. Handwriting Recognition

- Utilize advanced OCR technology for the recognition and digitization of handwritten mathematical symbols and equations.
- Achieve a competitive accuracy level in symbol and equation interpretation, aiming for state-of-the-art performance.

2. Mathematical Problem Solving

- Develop Natural Language Processing (NLP) algorithms to interpret and solve mathematical problems across multiple domains.
- Initial focus: Algebraic equations, basic calculus, and simplifications.

3. User Interface Design

- Create an intuitive, user-friendly interface supporting seamless input and interaction.

4. Performance and Reliability

- Ensure high recognition accuracy and solution reliability.
- Optimize computational efficiency.
- Minimize response time.

1.4 Scope

The Handwritten Mathematics Problem Solver project offers a broad and impactful scope, addressing critical challenges in mathematical learning, teaching, and professional problem-solving. Leveraging advanced machine learning and deep learning techniques, the project aims to create an intelligent system capable of interpreting and solving handwritten mathematical expressions with precision and efficiency. The scope encompasses both the development of a specialized neural network model and its integration into various educational and professional environments.

This project extends beyond traditional OCR systems, which are often limited in handling complex mathematical notations. By designing a tailored deep learning framework, the system will recognize and interpret diverse mathematical domains, including arithmetic, algebra, calculus, and trigonometry. It will transform handwritten inputs into accurate digital solutions, providing users with step-by-step explanations to enhance understanding and facilitate learning.

Ultimately, the project envisions a transformative impact on how handwritten mathematics is approached, making mathematical problem-solving more accessible, efficient, and interactive. Through this comprehensive solution, the project will bridge the gap between traditional handwritten methods and advanced computational intelligence, empowering users across academic, professional, and research domains.

2. Literature Review

2.1 Related Work

Mathematical problem-solving technologies have seen significant advancements, resulting in diverse applications aimed at automating the processing and solution of mathematical problems. This section critically examines several notable solutions in the handwritten mathematical problem-solving domain, highlighting their strengths and limitations concerning the challenges addressed by our project.

1. **Socratic:** Developed by Google, Socratic utilizes smartphone cameras to recognize and provide solutions to handwritten mathematical problems [2]. It offers educational step-by-step explanations. However, Socratic’s effectiveness is limited by its inconsistent recognition of complex mathematical notation and symbols, particularly those involving non-standard formatting or spatial arrangements. The system’s image-based recognition lacks the robust structural analysis needed to accurately parse complex expressions.
2. **Mathway:** An established online math solver, Mathway supports a broad range of mathematical topics, including algebra, calculus, and statistics [3]. It allows users to input equations via photo or text and provides detailed solution steps. Despite its breadth, Mathway struggles with highly cursive handwriting and complex equation layouts, revealing limitations in handling variations in handwriting styles and intricate formatting that are common in handwritten mathematical expressions.
3. **Photomath:** A widely used mobile application, Photomath boasts over 300 million downloads and offers real-time, camera-based solutions to mathematical problems [4]. It displays animated solution steps to enhance understanding. Nevertheless, Photomath’s performance is highly sensitive to image quality, decreasing significantly under poor lighting conditions or when processing messy handwriting [5]. This sensitivity underscores the challenges in robustly handling real-world input variations.
4. **Gauthmath:** Gauthmath combines AI-driven recognition with human expert verification to solve mathematical problems, particularly in advanced mathematical domains [6]. While the expert verification enhances accuracy, it introduces slower response times and the potential for subjective misinterpretation of unconventional notations. This

highlights the trade-off between automation and accuracy in complex mathematical problem-solving.

These existing applications have demonstrated progress in automating mathematical problem-solving. However, they face persistent challenges related to the variability in handwriting styles, the complexity of mathematical domains, and the need for intuitive solutions for advanced problems. Our project seeks to address these limitations by integrating advanced techniques for structural analysis, symbolic computation, and user interaction to provide a more robust and user-friendly solution.

2.2 Related Theory

2.2.1 Optical Character Recognition for Mathematical Notation

The recognition of handwritten mathematical expressions presents unique challenges that extend beyond those encountered in traditional text OCR. While standard OCR focuses on linear text strings, mathematical notation involves a complex two-dimensional structure with spatial relationships (e.g., superscripts, subscripts, fractions) that carry critical semantic meaning. These spatial relationships, crucial for the correct interpretation of expressions, are often difficult for standard OCR engines to process effectively. Several theoretical frameworks and techniques have been developed to address these specific challenges.

Structural Analysis Techniques:

Approaches based on structural analysis attempt to decompose mathematical expressions into their constituent elements and then analyze the spatial relationships between these elements. Grammars and graph-based methods are commonly used to represent the syntactic structure of mathematical expressions and guide the recognition process. These techniques are often combined with OCR engines to improve accuracy and robustness.

TrOCR Framework

TrOCR (Transformer-based OCR) signifies a substantial advancement in the realm of handwritten text recognition [1]. Unlike traditional OCR methods that rely on convolutional neural networks (CNNs), TrOCR harnesses the transformer architecture, which can capture long-range dependencies and contextual information within images:

- **Encoder-Decoder Architecture:** TrOCR employs a vision transformer encoder to process the input image, coupled with a text transformer decoder to generate the corresponding text output. This architecture facilitates a seamless bridge between visual and textual domains, enabling effective translation of images into structured text.

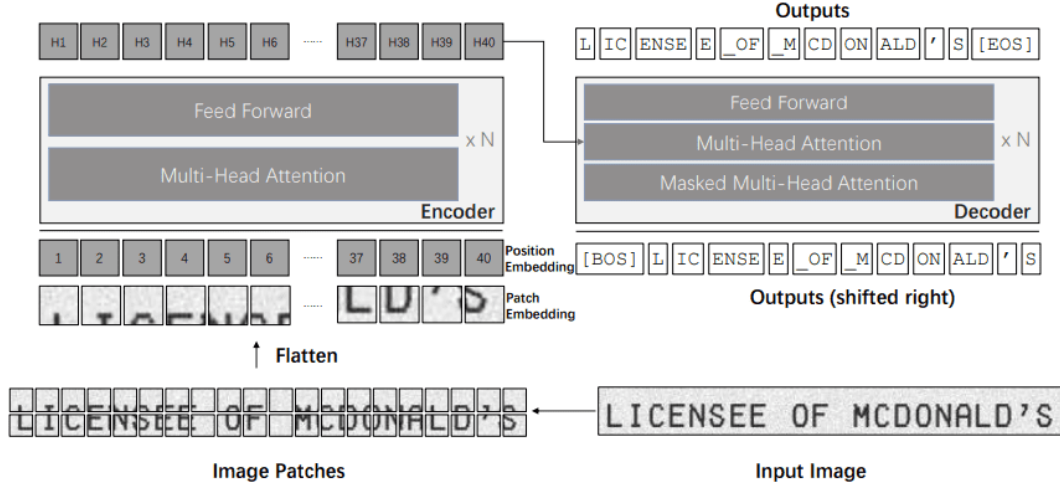


Figure 2.1: TrOCR Architecture Taken From [1]

- **Attention Mechanisms:** The self-attention mechanisms inherent in transformers enable the model to focus on salient parts of the input image while generating each output token. This is exceptionally valuable for understanding the spatial relationships and hierarchical structure that characterize mathematical notation.
- **Transfer Learning Capabilities:** TrOCR benefits significantly from pre-training on extensive datasets, allowing for the effective transfer of learned knowledge to the specialized domain of mathematical notation. This accelerates training and improves the model's generalization capabilities.
- **Context Awareness:** In contrast to traditional OCR systems that recognize individual symbols in isolation, TrOCR's transformer architecture maintains contextual awareness across the entire expression. This holistic approach enhances accuracy in deciphering complex mathematical structures by considering the relationships between symbols.

2.2.2 Mathematical Expression Parsing and Evaluation

After mathematical expressions are recognized, they must be parsed to create a structured representation that can be understood by a computer algebra system. This structured representation then allows for the evaluation and solution of the mathematical problem. This process involves several theoretical components, including parsing techniques and computer algebra systems.

Parsing Techniques:

Various parsing techniques, such as context-free grammars and operator precedence parsing, can be used to create a hierarchical representation of mathematical expressions. These

techniques enable the system to understand the structure of the expression and apply the appropriate mathematical operations.

Computer Algebra Systems and SymPy

SymPy, an open-source Python library for symbolic mathematics, provides the theoretical foundation for solving recognized mathematical problems [7]. SymPy offers a comprehensive suite of tools for manipulating and solving mathematical expressions.

- **Symbolic Computation:** Unlike numerical computation, which approximates solutions, symbolic computation manipulates mathematical expressions in their symbolic form, preserving exact relationships and enabling analytical solutions. This is crucial for obtaining accurate and generalizable results.
- **Expression Trees:** SymPy represents mathematical expressions as abstract syntax trees (ASTs), where operators and functions form internal nodes, and variables and constants form leaf nodes. This tree structure facilitates manipulation and transformation of expressions, allowing for symbolic simplification, differentiation, integration, and other operations.
- **Mathematical Domain Knowledge:** SymPy incorporates extensive domain-specific knowledge about mathematical operations, including algebraic simplification, calculus operations (differentiation, integration), equation solving, and more. This knowledge enables the system to perform complex mathematical tasks automatically.
- **Term Rewriting Systems:** The theoretical foundation of SymPy's simplification and transformation operations lies in term rewriting systems, which apply pattern-matching and replacement rules to transform expressions into equivalent, often simpler forms [8]. These systems provide a powerful and flexible way to automate mathematical manipulations.

Other relevant CAS: While SymPy is an excellent tool for symbolic calculations, it is not the only one, with other important options like Mathematica, Maple and SageMath offering alternative approaches to symbolic computation.

3. Methodology

The methodology for this project follows a structured approach to develop a robust hand-written mathematical expression recognition system. The process is divided into sequential phases, each focusing on a specific aspect of the system's development.

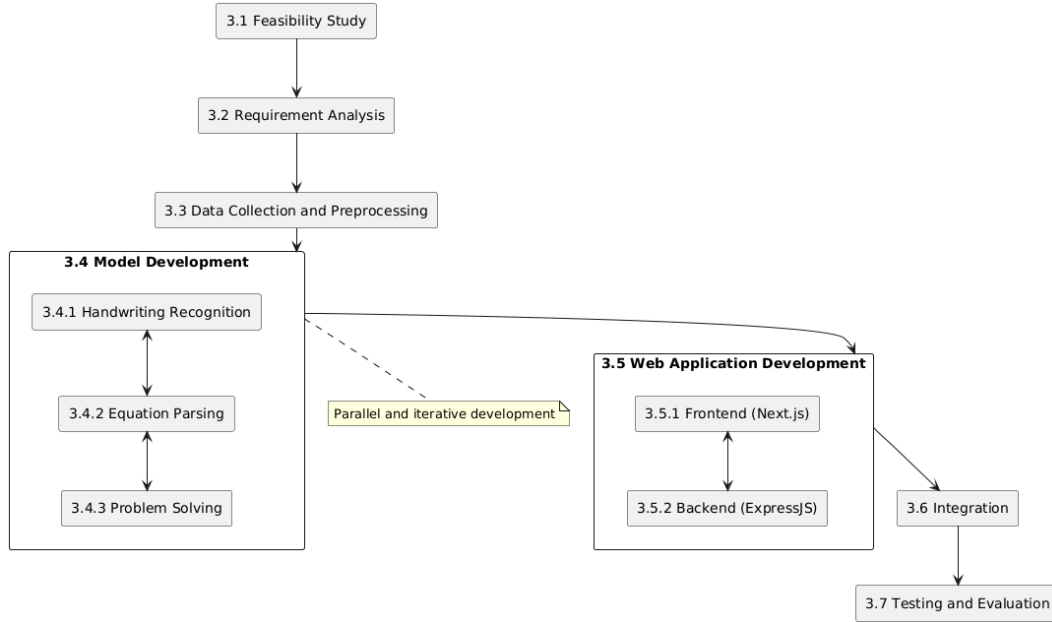


Figure 3.1: Methodology - Handwritten Mathematical Problem Solver

3.1 Feasibility Study

In this phase, we assessed the feasibility of recognizing handwritten mathematical expressions using deep learning models. We analyzed available datasets, computational resources, and existing state-of-the-art models to determine the viability of our approach. Additionally, we explored challenges in recognizing handwritten mathematical symbols, such as variations in handwriting styles and the complexity of mathematical notations.

3.2 Requirement Analysis

The system requirements were categorized into functional and non-functional aspects. The functional requirements included the ability to recognize handwritten mathematical expressions accurately, convert them into structured symbolic representations, and solve equations. The non-functional requirements focused on model efficiency, scalability, and real-time pro-

cessing capabilities. We also identified hardware and software requirements, ensuring compatibility with available computational resources.

3.3 Data Collection and Preprocessing

For training and evaluation, we used the CROHME 2014, 2016, and 2019 datasets, which contain handwritten mathematical expressions in InkML format [9, 10, 11]. Since the dataset was not in a directly usable format, we performed the following preprocessing steps:

- Converted InkML files to JPG images using custom scripts to facilitate model training.
- Cleaned the dataset by removing corrupted or incomplete samples and ensuring uniform dimensions.
- Split the dataset into training (80%), validation (10%), and testing (10%) sets for effective model evaluation.

3.4 Model Development

The model development phase was divided into multiple parts and was conducted in parallel and iteratively. It was done in the following three steps:

3.4.1 Handwriting Recognition

For recognizing handwritten mathematical expressions, we fine-tuned Microsoft’s TrOCR model, a transformer-based OCR system [1]. The modifications included:

- Expanding the tokenizer vocabulary to incorporate mathematical symbols.
- Fine-tuning the encoder-decoder transformer architecture using the CROHME dataset.
- Optimizing hyperparameters, such as batch size, learning rate, and training epochs, for better accuracy.
- Implementing gradient checkpointing to improve memory efficiency during training.
- Using WandB for logging and performance monitoring throughout the training process.

3.4.2 Equation Parsing

Once the handwritten expressions were recognized, the extracted text was converted into structured mathematical representations. We utilized SymPy, a Python library for symbolic computation, to handle the parsing process [7]. The steps involved:

- Converting recognized expressions into LaTeX format.
- Using SymPy’s parsing utilities to transform text-based equations into executable symbolic representations.
- Implementing custom grammar rules to resolve ambiguities in the recognition output.

3.4.3 Problem Solving

After parsing the mathematical expressions, the system was designed to solve equations using SymPy’s computational capabilities [7]. The functionalities included:

- Equation solving using `sympy.solve()` for algebraic computations.
- Differentiation and integration using `sympy.diff()` and `sympy.integrate()` for calculus-related tasks.
- Expression simplification using `sympy.simplify()` to reduce complex mathematical expressions.

3.5 Development of the Web Application

The web application was developed to provide an interactive interface for users to input handwritten mathematical expressions and obtain solutions. The system was divided into two main components: frontend and backend.

3.5.1 Frontend Development

The frontend was implemented using Next.js, leveraging its server-side rendering capabilities for improved performance. The user interface was designed for an intuitive experience, allowing users to upload images of handwritten equations and view the recognized output. TailwindCSS was used for styling, ensuring a responsive design.

The key interface components included:

- Equation Input Interface: Allows users to upload images and crop them to include the required math expression.
- Recognition Output Display: Shows the extracted mathematical expression in LaTeX format.
- Solution Display: Presents the solved equation.

3.5.2 Backend Development

The backend was built using ExpressJS, handling model inference and equation processing. The backend functions included:

- Receiving user-submitted images and processing them through the TrOCR model for recognition.
- Parsing and solving equations using SymPy.
- Returning the recognized and solved expressions to the frontend in real-time.

3.6 Integration

The system components were integrated to ensure smooth interaction between the frontend, backend, and deep learning model. The REST API endpoints facilitated seamless communication, enabling real-time recognition and equation solving. Model inference was optimized to minimize latency, ensuring a smooth user experience.

3.7 Testing and Evaluation

To evaluate the system's performance, we measured:

- **Recognition Accuracy:** Assessed how accurately the model transcribed handwritten mathematical expressions.
- **Parsing Success Rate:** Evaluated the correctness of equation formatting after parsing.
- **Solution Accuracy:** Verified whether the system-produced solutions matched expected results.

During training, model performance was continuously monitored using WandB, tracking metrics such as loss reduction and accuracy improvement. The system was tested with both seen and unseen handwritten equations to ensure robustness.

4. Experimental Setup

For our **Handwritten Mathematical Expression Recognition** system, the experimental setup plays a crucial role in ensuring the effectiveness of data collection, preprocessing, model training, and evaluation. This section details our data collection strategy, model training pipeline, and performance metrics.

4.1 Data Collection and Preprocessing

To build a robust dataset for handwritten mathematical expression recognition, we utilized the **CROHME (2014, 2016, 2019)** datasets. These datasets contain handwritten mathematical expressions in InkML format, which required conversion to a more accessible format for deep learning-based model training.

4.1.1 InkML to Image Conversion

Since the dataset was in InkML format, we converted each handwritten expression into an image format (JPG/PNG) using custom scripts and preprocessing libraries. This step was necessary to enable compatibility with deep learning models trained on image data.

4.1.2 Data Cleaning and Augmentation

To enhance the generalizability of our model, we applied the following preprocessing and augmentation techniques:

- **Image Normalization:** Standardized pixel intensity values for consistent model input.
- **Noise Reduction:** Removed artifacts and background noise using OpenCV filters.
- **Data Augmentation:** Applied *scaling, rotation, translation, and Gaussian noise addition* to increase dataset diversity.

4.1.3 Dataset Splitting

The dataset was divided into three subsets:

- **Training Set (80%)** – Used for model training.
- **Validation Set (10%)** – Used for hyperparameter tuning.

- **Test Set (10%)** – Used for final evaluation.

This division ensured a balanced distribution for robust model performance assessment.

4.2 Model Training Pipeline

To recognize handwritten mathematical expressions, we fine-tuned **Microsoft’s TrOCR model**, a transformer-based OCR model pre-trained on handwritten text. The model training was conducted using **GPU acceleration** to improve efficiency.

4.2.1 Training Environment

We leveraged an **NVIDIA RTX 3060 GPU Laptop** for local training and experimented with cloud-based resources such as **Google Colab** and **Kaggle’s Tesla T4 GPU**.

The following configurations were used for training:

- **Batch Size:** 8
- **Gradient Accumulation Steps:** 4
- **Effective Batch size:** $8 \times 4 = 32$
- **Optimizer:** AdamW
- **Learning Rate:** $5e^{-5}$ (default for AdamW in Seq2SeqTrainer)
- **Epochs:** 50

The model weights were periodically saved to **prevent overfitting** and to allow resumption from checkpoints.

4.2.2 Training Procedure

1. **Loading Pre-trained Processor:** A pre-trained processor (including tokenizer, image resizing/normalization) from "microsoft/trocr-base-handwritten" is loaded.
2. **Fine-tuning TrOCR:** The TrOCR encoder-decoder transformer model (pre-trained on stage1) is fine-tuned on the mathematical expression dataset using sequence-to-sequence learning. The model predicts the character sequence representing the mathematical expression given the input image.
3. **Model Saving and Checkpointing:** The model weights are saved periodically during training.
4. **Logging & Monitoring:** We integrated **Weights & Biases (WandB)** for real-time monitoring of loss curves and accuracy metrics.

4.2.3 Evaluation Metrics

To assess the performance of our system, we used multiple evaluation metrics:

Character Error Rate (CER)

Measures the number of incorrect characters in the predicted expression compared to the ground truth.

$$CER = \frac{\text{Substitutions} + \text{Insertions} + \text{Deletions}}{\text{Total Characters}} \quad (4.1)$$

Lower CER values indicate better performance.

Accuracy Rate

Measures the percentage of correctly predicted texts.

$$Accuracy = \frac{\text{Correct Predictions}}{\text{Total Predictions}} \quad (4.2)$$

5. System Design

The system design for the Handwritten Mathematical Problem Solver creates an end-to-end solution that transforms handwritten mathematical problems into solutions through a streamlined workflow.

5.1 Context Diagram

The context diagram (Figure 5.1) illustrates the high-level interaction flow of our handwritten mathematical problem solver system. The user provides input in the form of a handwritten mathematical inquiry, which is processed through several key components. The TrOCR model, trained on mathematical datasets, recognizes the handwritten input (labeled as RQ - Recognition Query) and converts it into LaTeX format. This structured mathematical representation is then passed to the Math Solver component, which utilizes SymPy to generate the solution. The final solution is returned to the user, completing the interaction loop. This streamlined workflow enables users to receive accurate solutions to their handwritten mathematical problems without needing to manually input complex expressions.

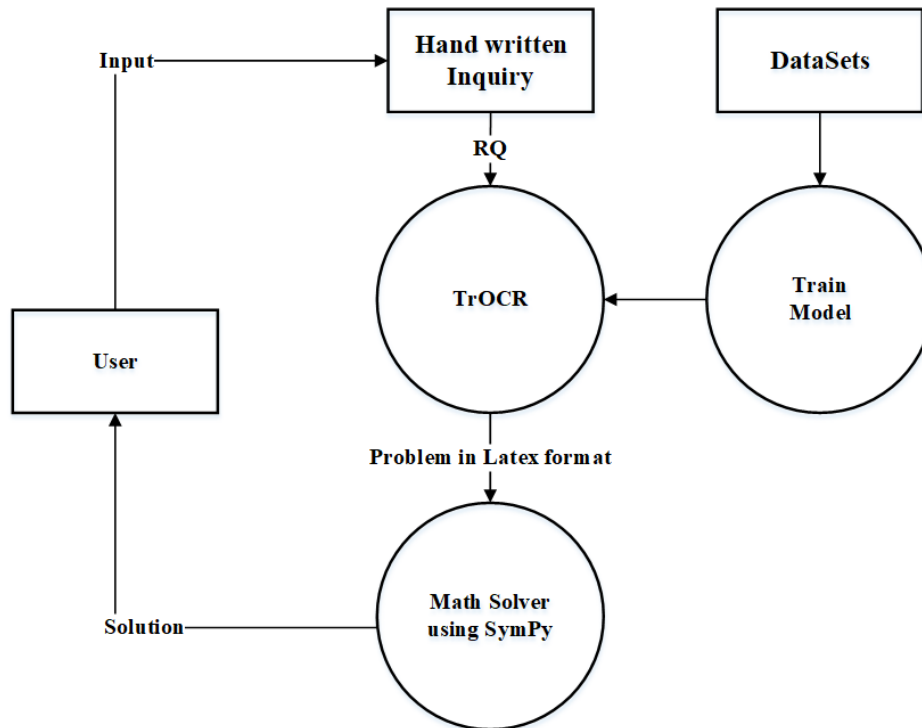


Figure 5.1: Context Diagram for Handwritten Mathematical Problem Solver

5.2 Use Case Diagram

5.2.1 Figure

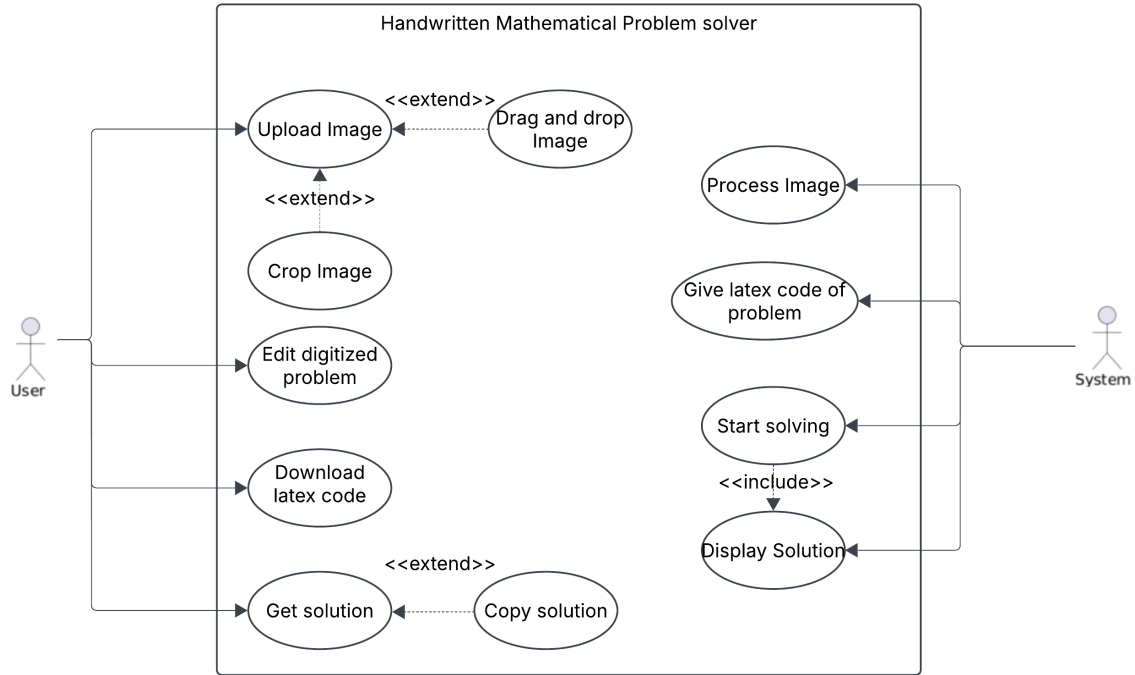


Figure 5.2: Use Case Diagram for Handwritten Mathematical Problem Solver

5.2.2 Description

The Use Case Diagram for the Handwritten Mathematical Problem Solver illustrates how users interact with the system to upload, process, and solve handwritten mathematical problems.

Use Cases

- **Upload Image:** The user uploads an image of a handwritten mathematical problem. This use case is extended by:
 - **Drag and Drop Image:** The user can alternatively drag and drop an image instead of manually selecting one.
 - **Crop Image:** After uploading, the user can crop the image to focus on the relevant mathematical expression.

- **Edit Digitized Problem:** The user can modify the recognized mathematical expressions to correct any errors in conversion.
- **Process Image:** The system processes the uploaded image, extracts handwritten text, and converts it into digital format.
- **Give LaTeX Code of Problem:** The extracted mathematical expression is provided in LaTeX format, allowing users to use it elsewhere.
- **Download LaTeX Code:** Users can download the LaTeX representation of the problem for documentation or further use.
- **Start Solving:** The system begins solving the mathematical expression extracted from the image. This use case includes:
 - **Display Solution:** The computed solution is shown to the user.
- **Get Solution:** The user can retrieve the final solution to the problem. This use case is extended by:
 - **Copy Solution:** The user can copy the solution to use elsewhere.

This diagram effectively showcases the system’s functionalities and the interaction between the user and the system to digitize and solve handwritten mathematical problems.

5.3 System Architecture

The detailed system architecture (Figure 5.3) expands on the context diagram by breaking down the process into three interconnected modules:

5.3.1 Model Training Module

This module focuses on preparing the TrOCR transformer model to recognize mathematical expressions.

- **Training Data:** Collection of image-LaTeX pairs for mathematical expressions
- **TrOCR Architecture:** Vision Transformer (ViT) encoder for image processing coupled with Text Transformer decoder for LaTeX generation
- **Fine-tuning:** Domain-specific adaptation for mathematical symbols and structures
- **Evaluation:** Performance assessment using recognition accuracy metrics

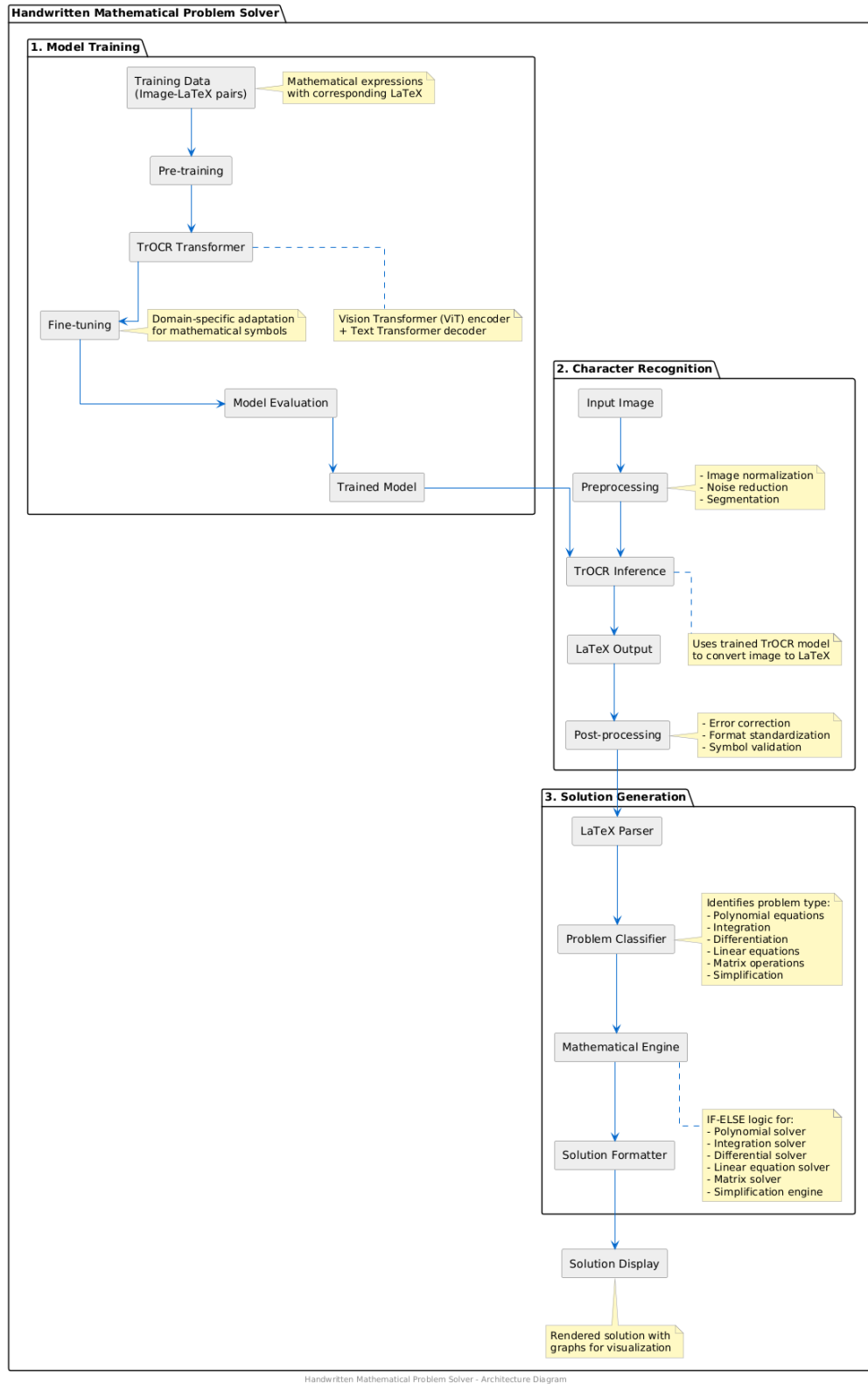


Figure 5.3: Detailed System Architecture of the Handwritten Mathematical Problem Solver

5.3.2 Character Recognition Module

When a user submits a handwritten problem, this module processes the input image through multiple processing steps.

- **Preprocessing:** Image normalization, noise reduction, and segmentation
- **TrOCR Inference:** Utilizing the trained model to convert handwritten input to LaTeX
- **Post-processing:** Error correction, format standardization, and symbol validation

5.3.3 Solution Generation Module

The final module parses the LaTeX expression, classifies the problem type (polynomial equations, integration, differentiation, etc.), and routes it to the appropriate mathematical engine.

- **LaTeX Parser:** Converts LaTeX into computational representation
- **Problem Classifier:** Categorizes expressions into problem types (equations, calculus, matrices)
- **Mathematical Engine:** Employs specialized solvers based on problem classification
- **Solution Display:** Presents formatted solutions with visualizations where applicable

6. Results & Discussion

6.1 Transformation of Handwritten Expressions to solution

The following are the results for different correct and incorrect recognition of math expressions and the solutions to the recognized latex problems.



Figure 6.1: Integration Problem: Handwritten, Recognized, and Solution



Figure 6.2: Integration Problem: Handwritten, Recognized, and Solution

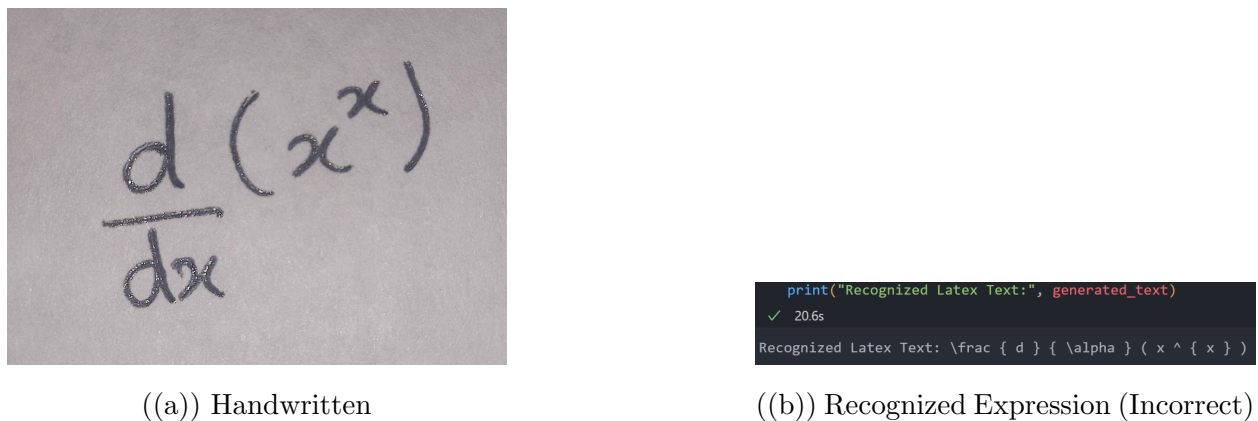


Figure 6.3: Derivative Problem: Handwritten and Recognized

6.2 User Interface

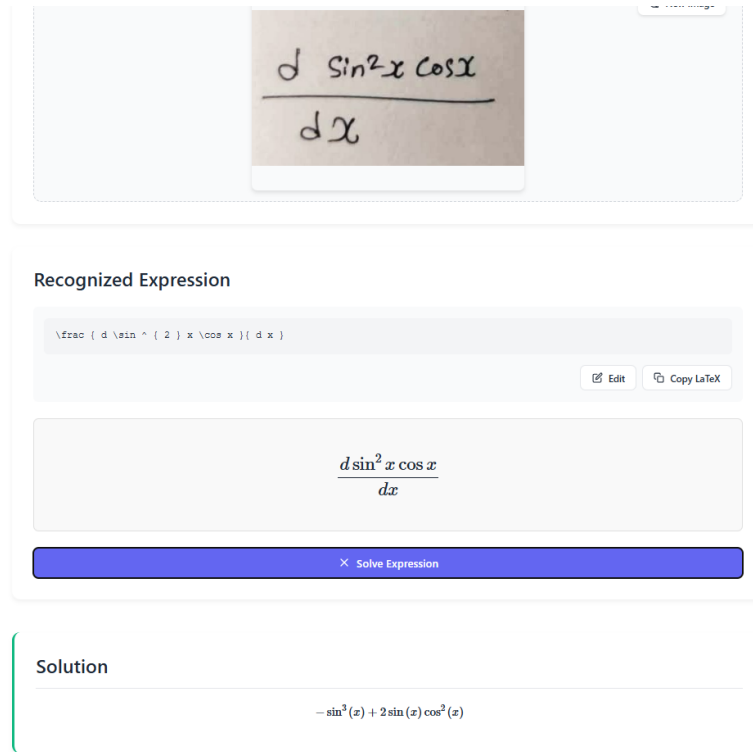


Figure 6.4: UI of the Web Application

6.3 Visualization

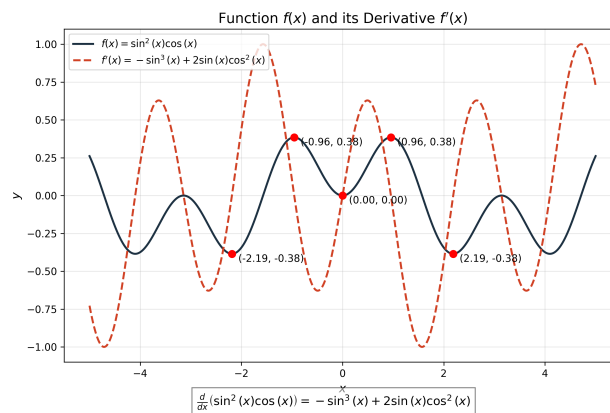


Figure 6.5: Graph visualization of problem provided in user interface

6.4 Statistical Analysis and Calculations

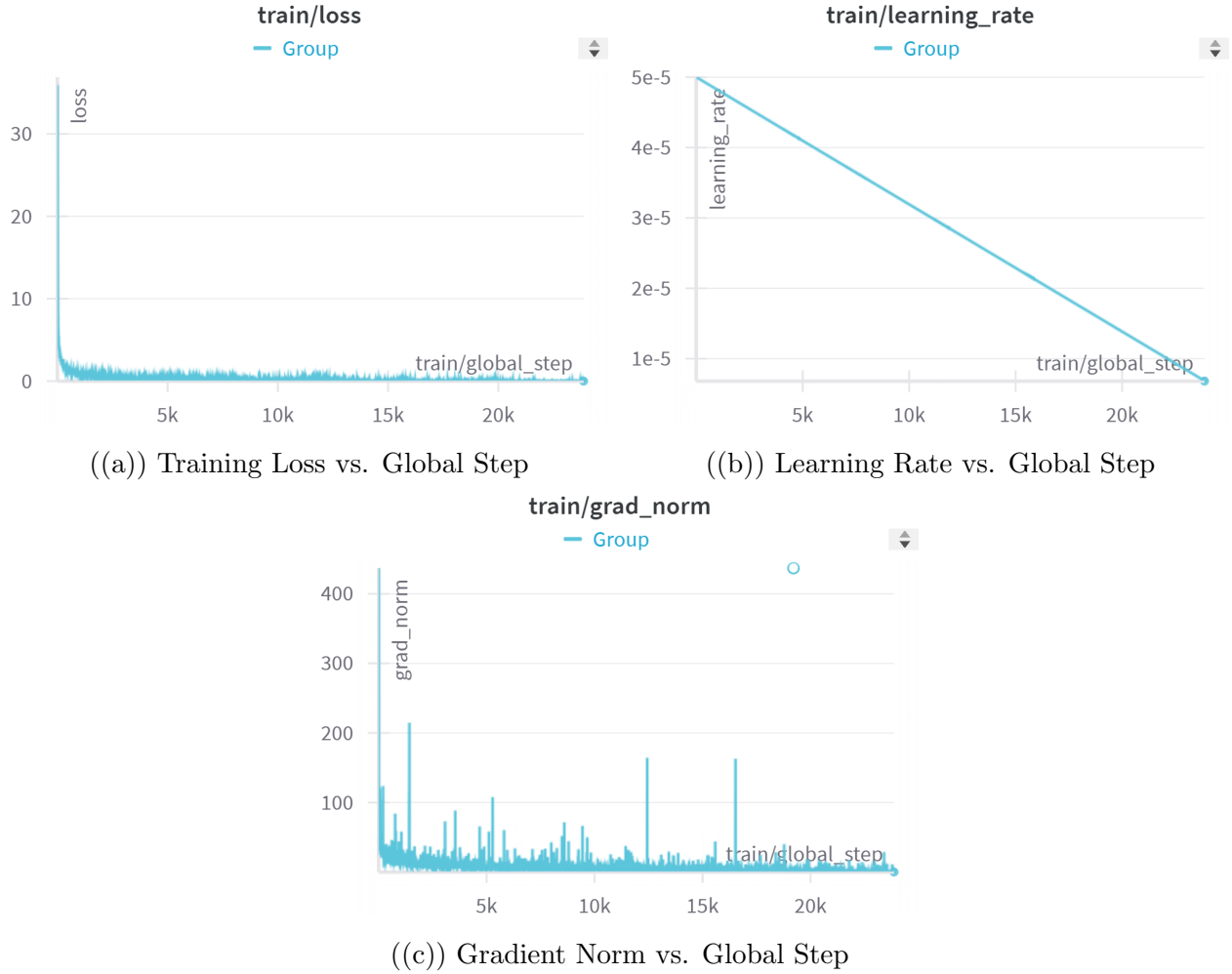


Figure 6.6: Training process visualization: (a) shows the loss decreasing over training steps, (b) depicts the learning rate schedule, and (c) illustrates gradient norm variations.

The figure illustrates key training metrics for the mathematical model:

- **Training Loss vs. Global Step** (Subfigure (a)): The loss starts high but rapidly decreases, indicating successful optimization. It gradually stabilizes, suggesting that the model is converging.
- **Learning Rate vs. Global Step** (Subfigure (b)): The learning rate follows a linear decay schedule, decreasing steadily over training. This helps stabilize learning and prevents overshooting.
- **Gradient Norm vs. Global Step** (Subfigure (c)): The gradient norm fluctuates

significantly at the start but stabilizes over time. Spikes indicate moments of large updates, which might be due to weight adjustments or noise in gradients.

6.5 Test Results

The handwritten math expression model was tested on samples from three different CROHME datasets i.e on 2014, 2016 and on 2019 datasets.

Table 6.1: Test Results on CROHME Datasets

Dataset	Number Samples	of Expression Recognition Accuracy (%)	Character Error Rate (%)
CROHME 2014	1199	22.515	30.701
CROHME 2016	1147	24.832	28.347
CROHME 2019	1147	23.801	29.703

6.6 Inference Time

The time required for the model to compute predictions is a crucial factor in real-time handwritten mathematical expression recognition. Our system currently exhibits an inference time ranging from **15 seconds to 1 minute**, depending on the complexity of the handwritten mathematical expression.

6.7 Discussion

The *Handwritten Mathematical Problem Solver* project marks a significant advancement in bridging the gap between handwritten mathematical notation and automated digital solutions. The integration of *TrOCR* for handwriting recognition and *SymPy* for symbolic computation has successfully enabled the system to process handwritten mathematical problems and provide accurate solutions. By converting handwritten mathematical expressions into LaTeX format, the system offers a streamlined, user-friendly interface that allows students, educators, and professionals to access solutions without manually inputting complex formulas.

One of the system’s key achievements is its ability to recognize and solve mathematical expressions. The use of *TrOCR*, a transformer-based optical character recognition model, ensures the effective conversion of handwritten text into machine-readable format. This has shown considerable success in recognizing a variety of algebraic and calculus-based problems, which are the most common types of mathematical expressions that users might encounter.

Through the application of *SymPy* as a symbolic computation engine, the system is capable of not only recognizing problems but also solving them with a high degree of accuracy.

An important aspect of the evaluation was **inference time**, which plays a crucial role in real-time handwritten mathematical expression recognition. The system currently exhibits an inference time ranging from **15 seconds to 1 minute**, depending on the complexity of the handwritten input. Simpler expressions are processed more quickly, while complex mathematical equations require additional time due to higher computational demands. This trade-off between accuracy and speed is a key consideration for future optimizations.

7. Conclusion

The Handwritten Mathematical Problem Solver project demonstrates significant progress in automated problem-solving systems. By integrating TrOCR for handwriting recognition and SymPy for symbolic computation, the system efficiently processes and solves mathematical problems from handwritten input. Its ability to recognize expressions, convert them into LaTeX, and solve basic problems makes it a valuable tool for education and professional use.

However, the system faces challenges with recognizing complex handwriting, advanced notations, and providing step-by-step solutions. These can be addressed with future enhancements, such as expanding the training dataset and supporting more advanced topics.

Future improvements could include real-time learning, refined error detection, and a more intuitive user interface. With continued development, this project has the potential to empower students, educators, and professionals by offering an accessible and efficient tool for solving mathematical problems.

In conclusion, the Handwritten Mathematical Problem Solver lays a strong foundation for the future of mathematical problem-solving, with promising potential for integration into education and professional settings.

8. Limitations

The Handwritten Mathematical Problem Solver system, while providing valuable assistance in solving mathematical problems, has several inherent limitations. These limitations are mainly due to the design and scope of the current system and must be considered when evaluating its effectiveness. Below are the key limitations categorized by the two primary sections of the system: the detection section and the solving section.

8.1 Limitations of the Detection Section

The detection section of the system, responsible for recognizing and interpreting the user's handwritten input, faces several challenges:

- **Inability to Recognize Complex Mathematical Symbols:** The detection section is optimized for basic mathematical symbols and expressions but may struggle with recognizing advanced symbols such as those used in higher-level calculus, trigonometry, and linear algebra. This limitation reduces the system's ability to solve more complex problems that involve specialized notations.
- **Dependency on Legible Handwriting:** The effectiveness of the system heavily depends on the legibility and clarity of the input. If the user's handwriting is ambiguous or poorly written, the system may misinterpret or fail to recognize the problem altogether, leading to inaccurate results or errors in the detected problem statement.
- **Limited Error Detection and Feedback:** The system currently lacks an error-detection mechanism that can flag unclear or incorrectly written problems. As a result, users may unknowingly submit problems with mistakes in their formatting, which the system cannot correct automatically.
- **Limited Handling of Non-Standard Input:** The detection system may not perform well when the input deviates from expected formats, such as writing complex mathematical expressions in non-standard ways or mixing textual descriptions with symbols.
- **Background and Foreground Noises:** The background noises like lighting effect of picture, smoothness of surface where expression is written has accountable impact on image to latex conversion. On other side problem detection also depends on ink color of text in foreground.

8.2 Limitations of the Solving Section

The solving section of the system focuses on processing the recognized mathematical problem and generating a solution. Despite its ability to solve certain mathematical problems, the following limitations need to be addressed:

- **Limited Scope of Solvable Problems:** The system is currently designed to solve only basic differentiation and integration problems, as well as simple polynomial equations and systems of linear equations. This means that more advanced topics, such as multi-variable calculus, differential equations of higher order, and matrix computations, are not supported by the current version of the system.
- **Lack of Step-by-Step Solutions:** A significant limitation of the solving section is that it does not provide step-by-step solutions. While the system can provide the final answer to problems, it does not guide users through the solution process. This can hinder users' learning, as they may not understand how the solution was derived or how to approach similar problems in the future.
- **No Support for Geometric or Arithmetic Problems:** The current system is not capable of solving geometric problems (such as calculating areas, volumes, or angles) or arithmetic problems (such as basic addition, subtraction, multiplication, and division), which limits its applicability for a broader range of mathematical challenges.
- **Inability to Handle Complex Problems:** More complex mathematical problems, such as multi-dimensional integrals, optimization problems, or non-linear equations, cannot be solved effectively by the system. The system may fail to recognize such problems or provide incorrect results for them.
- **No Error Checking for Solution Accuracy:** The system does not currently perform any internal checks to verify the accuracy of the solution it generates. As a result, there is a possibility that the system could provide incorrect or incomplete solutions without flagging any errors to the user.

9. Future Enhancements

In order to address the current limitations and extend the functionality of the Handwritten Mathematical Problem Solver system, several future enhancements can be considered. These improvements can help enhance both the detection and solving capabilities of the system, making it more robust and capable of solving a broader range of mathematical problems. Below are the proposed future enhancements.

9.1 Future Enhancements to the Detection Section

The detection section plays a critical role in the accuracy and usability of the system. Enhancing this section will improve the overall user experience. Proposed improvements include:

- **Advanced Symbol Recognition:** To address the limitations in recognizing complex mathematical symbols, the system can be enhanced to recognize a wider array of advanced mathematical notations, including those used in higher-level calculus, matrix operations, and trigonometry. This would allow the system to handle a broader spectrum of problems, including those involving specialized symbols.
- **Improved Error Detection and Feedback:** Incorporating an error detection system that can analyze and flag ambiguous or incorrectly written problems would significantly improve the user experience. For example, if a user submits an incomplete or unclear problem, the system could alert the user and ask for clarification before attempting to solve it.
- **Multi Line Expression Detection and Solution:** Adding feature of multi line expression detection advances our current single line constraint to ability of detecting and solving problems like system of linear equations and matrix related problem domains.
- **Support for Mixed Input (Text and Symbols):** Future versions of the system could incorporate natural language processing (NLP) techniques to handle problems where mathematical expressions are mixed with text. This would allow the system to process word problems more effectively by converting textual descriptions into mathematical formulas.

9.2 Future Enhancements to the Solving Section

The solving section is crucial for generating accurate and comprehensive solutions. Enhancements to this section will improve the system's problem-solving capabilities. Proposed improvements include:

- **Support for Advanced Mathematical Topics:** Expanding the system's ability to solve advanced problems such as multi-variable calculus, higher-order differential equations, and matrix operations would significantly increase its utility. By adding support for these topics, the system can become a more powerful tool for students and professionals dealing with complex mathematical problems.
- **Step-by-Step Solution Generation:** Adding the capability to provide step-by-step solutions would enhance the educational value of the system. By guiding users through the solution process, the system would help users understand the steps involved in solving each problem. This could be particularly useful for students learning new mathematical concepts and for anyone looking to understand how to solve similar problems in the future.
- **Support for Geometric and Arithmetic Problems:** Extending the system's capabilities to include geometric problem-solving (such as calculating areas, volumes, angles, etc.) and basic arithmetic operations would make the system more versatile and applicable to a wider range of mathematical disciplines.
- **Enhanced Handling of Complex Problems:** The system should be enhanced to handle more complex problems, such as multi-dimensional integrals, optimization problems, and non-linear equations. Improved algorithms and more advanced solving techniques would enable the system to provide accurate solutions to these types of problems.
- **Real-Time Learning and Adaptation:** Implementing real-time learning would allow the system to improve continuously based on user feedback. If users provide corrections or modifications to the system's output, the system could learn from these corrections to provide more accurate solutions in the future.
- **Performance Optimization for Large Problems:** Optimizing the system to handle larger and more complex problems efficiently would be important for enhancing performance. This could include optimizing algorithms for faster processing and reducing the time it takes to solve high-complexity problems.

- **Enhanced User Interface:** Future enhancements could also include improvements to the user interface. Features such as drag-and-drop problem submission, voice input recognition, and real-time feedback on the solutions would make the system more user-friendly and accessible to a broader audience.

References

- [1] Minghao Li, Tengchao Lv, Jingye Chen, Lei Cui, Yijuan Lu, Dinei Florencio, Cha Zhang, Zhoujun Li, and Furu Wei. Trocr: Transformer-based optical character recognition with pre-trained models, 2022.
- [2] Socratic by google.
- [3] Mathway: Solve your math problems.
- [4] Photomath.
- [5] Nihar Cherukuri, Brandon Edwards, and Joseph Thornton. Photomath goes to college: an evaluation of its ability to solve problems from introductory calculus. *PRIMUS*, 28(8):739–753, 2018.
- [6] Gauthmath - homework helper.
- [7] SymPy Development Team. Sympy: Python library for symbolic mathematics, 2024.
- [8] Franz Baader and Tobias Nipkow. *Term Rewriting and All That*. Cambridge University Press, 1998.
- [9] Andreas Striegel, Harold Mouchère, Christian Viard-Gaudin, Jin-Hyuk Kim, and Richard Zanibbi. Icfhr 2016 competition on recognition of online handwritten mathematical expressions (crohme). In *2016 15th International Conference on Frontiers in Handwriting Recognition (ICFHR)*, pages 577–582. IEEE, 2016.
- [10] Harold Mouchère, Christian Viard-Gaudin, Richard Zanibbi, Utpal Garain, Jin-Hyuk Kim, and Tracy Miller. Icfhr 2014 competition on recognition of online handwritten mathematical expressions (crohme 2014). In *2014 International Conference on Frontiers in Handwriting Recognition (ICFHR)*, pages 607–612. IEEE, 2014.
- [11] Samira Mahmoudi, Harold Mouchère, Mauricio Villegas Sepúlveda, Christian Viard-Gaudin, Richard Zanibbi, and Jin-Hyuk Kim. Crohme + tpie 2019: Competition on recognition of handwritten mathematical expressions and typeset pdf images of equations. In *2019 International Conference on Document Analysis and Recognition (ICDAR)*, pages 1536–1542. IEEE, 2019.